SQL Server → JSON migration plan (Kelly Tools For Revit)

Goal: Kelly Tools For Revit (KTR) makes no RDBMS calls. Every read comes from JSON files in `C:\Program Files\JH Kelly\Kelly Tools For Revit\Data`.

Re-analysed 2026-10-21 against **this generation** of KTR. Kelly-Domain and Kelly-Revit-UI are not part of it. The generation is the checked-out working trees, including their local changes:

Repo	Branch	Kelly packages it pins
Kelly-Windows-Data	`26.10.02.1034`	–
Kelly-Windows	`26.10.02.1034`	Kelly.Windows.Data 26.10.21.807/808
Kelly-Revit-Logic	`main-Gen4`	Kelly.Windows 26.10.21.816, Kelly.Windows.Data
Kelly-Revit	`main-Gen4`	Kelly.Revit.Logic 26.10.21.817/818, Kelly.Windows, Kelly.Windows.Data

`develop` in these repos belongs to a different generation, which uses Kelly-Domain. The first version of this plan mixed findings from it; this version does not.

---

1. What KTR calls today

How the calls work:

- Every call is a **stored procedure** run through Dapper in `Kelly.Windows.Data.Services.SqlServerService`, which takes a procedure name (`CommandType.StoredProcedure`). KTR has no inline SQL, no EF, and no OleDb, ODBC or SQLite.
- The code names **34 procedures**: 26 read, 7 write and 1 is dead. The tables below give the full list.
- There are two databases on `lvsqlp03\misc`:
  - **VCS**: `App.ConnectionStringGeneralData`, connection string `VirtualConstructionService`.
  - **VCS Content**: `App.ConnectionStringContentData`, connection string `VirtualConstructionServiceContent`.
- At startup `App.ResolveConnectionStrings` opens and closes both databases. `Kelly.Revit.dll.config` and `Settings.settings` hold two procedure names: `StoredProcedureCustomDataUpdaterSelectAll` and `StoredProcedureGaugeReadWhereEquals`.

Where the calls are:

Repo	Files that call SQL
Kelly-Windows-Data	`Services/SqlServerService.cs`, `BaseClasses/BaseServerService.cs`, `Interfaces/IDatabaseDataService.cs` (the service itself, no calls of its own)

Repo	Files that call SQL
Kelly-Windows	`Services/TimerService.cs` (dead: declared `file class`), and `Interfaces/IModel.cs`, whose `Record*` members take `IDatabaseDataService`
Kelly-Revit-Logic	`Models/ExternalParameterDefinition.cs`, `Models/GroupTypeIdWrapper.cs`, `Models/ProjectInformationWrapper.cs`
Kelly-Revit	everything else, listed in the tables below. `modules/` holds no source in this generation, and every command is under `src/Documents/Commands`.

Already off SQL: `CategoryType.json` is loaded by `App.CacheCategoryTypeRecords`. `OperationService` (Logic) appends its audit log to a local JSONL file, which is the model for decision D1 below. `SpoolManagerViewModel` reads the Terminology cache.

Checked and found clean: Kelly-Revit's `SQL Server Service - Usage Recommendations.csv` also lists `CategoryExportToSqlServerViewModel`, `DetailerService`, `FabricationPartDataExporterViewModel`, `RibbonTools`, `RibbonTabEntityExtensions` and `SystemParameterCache`. None of these exists in this generation, apart from a commented-out `DetailerService` line.

Reads (26)

Procedure	DB	Caller (Kelly-Revit unless noted)	JSON file
`[dbo].[USP_ExternalParameterDefinition_SelectAll_v20240113]`	VCS	`Application/Caches/ExternalParameterDefinitionCache.cs:54`	ExternalParameterDefinition
`[revit].[USP_GroupTypeId_SelectWhere_v20250110]`	VCS	Logic `Models/GroupTypeIdWrapper.cs:38`	GroupTypeId
`[dbo].[USP_BuiltInCategory_SelectWhereDefinitionIdAndRevitVersionEquals]`	VCS	Logic `Models/ExternalParameterDefinition.cs:64`	BuiltInCategoryBinding
`[dbo].[USP_Table_ProductRange_ReadAll]`	VCS	`Application/Caches/ProductRangeCache.cs:35`	ProductRange
`[dbo].[USP_Table_ProductRangeParameterDefaults_ReadWhereProductRangeIdEqualsId]`	VCS	`ProductRangeCache.cs:55`	ProductRangeParameterDefaults
`[dbo].[USP_ServiceData_SelectAll_v20241010]`	VCS	`Application/Caches/ServiceCache.cs:35`	ServiceData
`[core].[USP_System_SelectAll_v20241010]`	VCS	`Application/Caches/SystemCache.cs:35`	System
`[core].[USP_System_SelectWhereIdEquals]`	VCS	`Documents/Caches/SystemDocumentCache.cs:90`	System (by Id)
`[core].[USP_System_SelectWhereNameAndAbbreviationEquals]`	VCS	`SystemDocumentCache.cs:131`	System (by Name + Abbreviation)
`[core].[USP_System_SelectWhereSignatureEquals]`	VCS	`Documents/Services/ElementServices/System/SystemUpdaterElementService.cs:47`	System (by `Name + " " + Abbreviation`)
`[dbo].[USP_Terminology_SelectAll]`	VCS	`Application/Caches/TerminologyCache.cs:35`	Terminology
`[core].[USP_ProjectToSystem_SelectWhereNumberAndConfigurationEquals]`	VCS	`SystemDocumentCache.cs:81`	ProjectToSystem
`[dbo].[USP_EndPreparationsWelded_Select]`	VCS	`Documents/Services/ElementServices/EndPreparation/EndPreparationUpdaterElementService.cs:49`	**EndPreparationsWelded** (added in this re-analysis)
`[dbo].[USP_ConnectorPreparation_Select]`	VCS	`EndPreparationUpdaterElementService.cs:93`	ConnectorPreparations
`[dbo].[USP_Project_SelectWhereFilePathAndFileStoreEquals]`	VCS	Logic `Models/ProjectInformationWrapper.cs:210`	Project
`[dbo].[USP_View_ComponentAndLookups_ReadAll]`	Content	`Application/Caches/ComponentCache.cs:35`	ComponentAndLookupsView
`[dbo].[USP_Table_Product_ReadAll]`	Content	`Application/Caches/ProductCache.cs:35`	Product
`[dbo].[USP_Table_ConnectorGrouping_ReadWhereIdEqualsParameter]`	Content	`Common/Modelle/ElementDataModels/SizeElementDataModel.cs:365`	ConnectorGrouping
`[dbo].[USP_ConnectorMapping_SelectWhereIdEqualsParameter_v20240716]`	Content	`SizeElementDataModel.cs:379`	ConnectorMapping
`[dbo].[USP_SizeSymbol_SelectWhereTradeIdEqualsParameter_v20240716]`	Content	`SizeElementDataModel.cs:395`	SizeSymbol
`USP_View_MaterialAndLookups_ReadWhereEquals`	Content	`Common/Modelle/DataTransferObjects/MaterialAndLookups.cs:88`, from `MaterialElementDataModel.cs:143`	MaterialAndLookupsView
`USP_Table_Gauge_ReadWhereEquals` (name from settings)	Content	`Common/EntityServices/GaugeNameResolver.cs:57` (ScheduleGauge updater, CustomData V2)	Gauge
`[fabrication].[USP_Configuration_SelectWhereGuidEquals]`	Content	`SystemDocumentCache.cs:61`	Configuration
`[dbo].[USP_CustomDataUpdater_SelectAll_v20260811]` (hard-coded and from settings)	Content	`Documents/Commands/Contenter/FabricationCustomDataPropsed/FabricationCustomDataReportCommand.cs:20`, `FabricationCustomDataUpdaterV2/FabricationCustomDataUpdaterCommandv2.cs:329`	CustomDataUpdater
`[fabrication].[USP_ItemCustomDataPivotedOnComponentIdView_SelectAll_v20260112]`	Content	`FabricationCustomDataUpdaterV1/FabricationCustomDataUpdaterCommandv1.cs:22`	ItemCustomDataPivotedOnComponentId
`[dbo].[USP_CategoryType_SelectAll_v20250221]`	Content	`Documents/Commands/Developer/ProductRangeSchedules/ProductRangeSchedulesCommand.cs:36`	CategoryType (already loaded from JSON at startup)

Writes (7 live)

Procedure	Table	Caller	When
`[dbo].[USP_Table_ApplicationSession_CreateOne]`	dbo.ApplicationSession	`Common/Modelle/DataTransferObjects/ApplicationSession.cs:120`, from `Application/Services/ApplicationSessionService.cs`	Revit startup
`[dbo].[USP_Table_ApplicationSession_UpdateWhereIdEqualsParameter]`	dbo.ApplicationSession	`ApplicationSession.cs:132`	Revit shutdown
`[dbo].[USP_ProjectSession_Insert]`	dbo.ProjectSession	`DocumentSession.cs:98`, from `Documents/Services/SessionServices/DocumentSessionService.cs:36`	DocumentOpened (**throws** `InvalidOperationException` **if the insert fails**)
`[dbo].[USP_ProjectSession_Update]`	dbo.ProjectSession	`DocumentSession.cs:144`	DocumentClosing
`[dbo].[USP_Table_ProjectMetricsLog_Insert]`	dbo.ProjectMetricsLog	`DocumentSession.cs:162`	DocumentClosing (time to open)
`[dbo].[USP_Project_Insert_v20241205]`	dbo.Project	Logic `ProjectInformationWrapper.cs:185`, from `Common/Werkzeuge/DocumentTools.cs:38`	DocumentOpened reconcile, when no row exists. The new Id is written into the model's `JhkProjectId`.
`[dbo].[USP_Project_UpdateWhereIdEquals]`	dbo.Project	Logic `ProjectInformationWrapper.cs:249`, from `DocumentTools.cs:66`	DocumentOpened reconcile

Dead, delete rather than convert:

- `[Development].[USP_CodeTimer_Insert_v20241112]` in Kelly-Windows `TimerService`, which is a `file class` and so unreachable.
- `DocumentSession.RecordInsertPrev`, which has no callers.
- `ServerDataService.CheckDatabaseServerConnections`, which is private and never called.

2. JSON files to export: 23, all produced by `Kelly-Data-Exporter`

Recommendation: export each procedure's **result set**, not the raw tables.

- The record types already map those columns by name, so the C# does not change shape.
- The big joins stay in SQL at export time instead of being rewritten in C#.
- Parameterized procedures are exported **without their parameters**, and the filter becomes an in-memory lookup.
- File names follow `JsonFileRepository`'s rule: the record type name without `Record`.

Status: **built, and exported into `Data\` on 2026-10-21 in the record-shaped form (23 files plus `manifest.json`).** Every file's hash matches the manifest. Every file also loads back with the manifest's row count: 22 through `JsonFileRepository<T>`, and `EndPreparationsWelded` through `JsonConvert`.

#	File	DB	Export source	Lookup key in the add-in
1	ExternalParameterDefinition	VCS	its SelectAll procedure	name (OrdinalIgnoreCase)
2	GroupTypeId	VCS	`revit.GroupTypeId`	Id
3	BuiltInCategoryBinding *(new record)*	VCS	the procedure's join, plus `DefinitionId` and `RevitVersion`	(DefinitionId, RevitVersion)
4	ProductRange	VCS	its ReadAll procedure	–
5	ProductRangeParameterDefaults	VCS	`dbo.ProductRangeParameterDefaults`	ProductRangeId
6	ServiceData	VCS	its SelectAll procedure	–
7	System	VCS	its SelectAll procedure	Id; (Name, Abbreviation); `Name + " " + Abbreviation`
8	Terminology	VCS	its SelectAll procedure	–
9	ProjectToSystem	VCS	`core.ProjectToSystem`	(ProjectNumber, ConfigurationId)
10	**EndPreparationsWelded** *(new record)*	VCS	the procedure's join, keeping its fixed `Weld` / `Location` filter, plus the linker `Id` (7 rows)	(WeldLocationInitialism, EndPreparationsHasFieldTrim)
11	ConnectorPreparations	VCS	the procedure's join, plus the connector `Id`	(GroupName, ConnectorName)
12	Project *(see D2)*	VCS	`dbo.Project`, the procedure's 25 columns	(FilePath, FileStore)
13	ComponentAndLookupsView	Content	its ReadAll procedure	–
14	Product	Content	its ReadAll procedure	–
15	ConnectorGrouping	Content	`dbo.ConnectorGrouping`	Id
16	ConnectorMapping	Content	`dbo.ConnectorMapping`	Id
17	SizeSymbol	Content	`dbo.SizeSymbol`	TradeId
18	Gauge	Content	`dbo.Gauge`	(MaterialTypeId, Thickness)
19	MaterialAndLookupsView	Content	the procedure's join	(Name, MaterialGroupDotName)
20	Configuration	Content	`Fabrication.Configuration` (confirmed: `Material.Configuration` has no Guid)	Guid
21	CustomDataUpdater	Content	its SelectAll procedure	–
22	ItemCustomDataPivotedOnComponentId	Content	its SelectAll procedure	–

#	File	DB	Export source	Lookup key in the add-in
23	CategoryType	Content	`dbo.CategoryType`, **not its procedure**: the procedure leaves out `IsValidForSpool` and `IsValidForStratus`, and `SpoolItem` reads `IsValidForSpool`	–

How the exports were checked:

- Each export that replaces a procedure returns the same columns as the live procedure. The only extras are the ones added on purpose: `DefinitionId`, `RevitVersion` and the two `Id` columns.
- A full export takes about 8 seconds and writes about 212 MB. `Product.json` and `CustomDataUpdater.json` are about 93 MB each.

---

3. Base tables behind those files (63)

VCS (19 tables):

- `dbo.ExternalParameterDefinitions`
- `revit.GroupTypeId`
- Categories: `dbo.BuiltInCategory`, `dbo.BuiltInCategoryMeta`, `dbo.LinkerExternalParameterDefinitionsToCategories`
- Product ranges: `dbo.ProductRange`, `dbo.ProductRangeParameterDefaults`
- `dbo.ServiceData`, `core.System`, `dbo.Terminology`, `core.ProjectToSystem`
- Welded end preparations: `dbo.Linker_WeldLocationToEndPreparationWelded`, `dbo.EndPreparationsWelded`
- Connectors: `dbo.Connector`, `dbo.ConnectorDomains`, `dbo.ConnectorShapes`, `dbo.ConnectorPreparations`, `dbo.ConnectorGroups`
- `dbo.Project`

VCS Content (44 tables):

- **Component lookups (29), all `dbo`:**
  - `Component`, `Business`, `BusinessModel`, `CatalogGroup`, `Category`, `CategoryType`, `ComponentGroup`, `ComponentModifier`, `ConnectionType`
  - `FabricationPattern`, `FabricationPatternOptionType`, `FileNameModifier`, `FileType`, `Finish`
  - `MainClass`, `MainClassModifier`, `MainSchedule`
  - `MaterialCode`, `MaterialGrade`, `MaterialGroup`, `MaterialSpecification`, `MaterialType`
  - `ReducingClass`, `ReducingClassModifier`, `ReducingSchedule`
  - `SubCategory`, `SubCategoryModifierAlpha`, `SubCategoryModifierBravo`, `Trade`
- **Custom data (6 more, `dbo`):** `ComponentSupportType`, `SupportType`, `ComponentTag`, `TagPrefix`, `ComponentWeldType`, `WeldType`
- **Others (9):**
  - `dbo`: `Product`, `ConnectorGrouping`, `ConnectorMapping`, `SizeSymbol`, `Gauge`
  - `Fabrication`: `Material`, `MaterialGroup`, `Configuration`, `ItemCustomData`

Written but never read: `dbo.ApplicationSession`, `dbo.ProjectSession` and `dbo.ProjectMetricsLog`. They need no export.

4. Writes: decisions needed

`Program Files` cannot be written by standard users, so the 7 write procedures need a decision.

Decision	Procedures	Options
D1 Session and metrics logging	ApplicationSession CreateOne/Update, ProjectSession Insert/Update, ProjectMetricsLog Insert	(a) drop all five; (b) append to `%LOCALAPPDATA%\JH Kelly\Kelly Tools For Revit\Sessions.jsonl` with `JsonLinesFileWriter`, the way `OperationService` already logs. The server Ids that link the records (`ApplicationSession.Id` → `ProjectSession.ApplicationSessionId`) become local GUIDs. Either way, `DocumentSessionService` must stop throwing when the insert "fails".
D2 Project reconcile	`USP_Project_Insert_v20241205`, `USP_Project_UpdateWhereIdEquals` (Logic `ProjectInformationWrapper`, called on every DocumentOpened)	(a) read-only: look up `Project.json`; a document with no row keeps its current info and gets no new Id; (b) drop the reconcile; (c) a per-user Id source (Ids would differ between users, so not recommended)

5. Steps

The work goes bottom-up through the four repos. Each follows the shared rules: TDD, the `Author`/`Description` attributes, the Changelog/README/SOLID updates, then pack, repin the layers above and rebuild.

Phase 0: prerequisites

1. **Branch.** This generation lives on `26.10.02.1034` and `main-Gen4`, but the shared rule says to work only on `develop`, and `develop` is the other generation. Decide which branch this work is committed to. All four trees also have 2–3 uncommitted changes.
2. Decide D1 and D2.
3. Confirm that SQL Server stays the master and the exporter is re-run (recommended), rather than the JSON becoming the master.

Phase 1: exporter. Done: `C:\repo\jhkweb\Kelly-Data-Exporter`, 86 tests. What's left:

1. Decide where it lives permanently (its own repo, or a folder in Kelly-Revit).
2. Decide how the snapshot reaches workstations: the installer or the auto-updater.

Phase 2: Kelly-Windows-Data

1. Read the files through the existing JSON service, `Services.JsonFileRepository<T>`, read only: `GetAll`, `Find`, `FirstOrDefault`, `GetById`, `Reload`. The add-in never writes them.
2. `LoadFromDisk` **catches** `JsonException` **and falls back to an empty store**, so a corrupt or mismatched file loads silently as zero rows. Make that visible for these files: a strict load option, or a startup check that compares `Count` with `manifest.json`'s row count.
3. Add the record types the files need. Move `BuiltInCategoryBindingRecord` here from the exporter,

and make `UspEndPreparationsWeldedSelectOne` an `IRecord` or replace it with `EndPreparationsWeldedRecord`. `ConnectorPreparationsRecord` already has `Id`.

4. Build Net481, Net80 and Net100.

Phase 3: Kelly-Windows

1. Delete `TimerService`.
2. Change `IModel`'s `Record*(IDatabaseDataService)` members. Their only real implementations are the session DTOs; the rest return false or throw `NotImplementedException`. Drop the members, or point them at the JSON source.

Phase 4: Kelly-Revit-Logic

1. `ExternalParameterDefinition`: bound categories from `BuiltInCategoryBinding.json`.
2. `GroupTypeIdWrapper`: from `GroupTypeId.json`.
3. `ProjectInformationWrapper`: per D2. Its constructor takes a connection string at `DocumentSession.cs:14`, `SystemDocumentCache.cs:72` and `DocumentTools.cs:20/29`.
4. Run the in-Revit tests on 2024 with the add-in disabled.

Phase 5: Kelly-Revit

1. `BaseCache` and the 7 caches (`ServerDataService.CacheServerData`): load from `Data\`. The two per-row lookups that run while the caches fill become in-memory joins: ProductRange defaults, and GroupTypeId/BuiltInCategory for each definition.
2. Element services and data models:
   - `GaugeNameResolver`
   - `SizeElementDataModel` (Grouping, Mapping, SizeSymbol)
   - `MaterialAndLookups.ServerSelect` / `MaterialElementDataModel`
   - `SystemDocumentCache` (Configuration, ProjectToSystem, System by Id and by Name + Abbreviation)
   - `SystemUpdaterElementService`
   - `EndPreparationUpdaterElementService` (welded and connector preparation)
3. Commands:
   - `FabricationCustomDataReportCommand` and `FabricationCustomDataUpdaterCommandV1`/`V2`
   - `ProductRangeSchedulesCommand`, which should use the CategoryType cache already loaded at startup
4. `ApplicationSession`, `ApplicationSessionService`, `DocumentSession`, `DocumentSessionService`: per D1.
5. `App.cs`: remove `ResolveConnectionStrings`, the connection test and `ConnectionStringGeneralData`/`ContentData`.
6. `Kelly.Revit.dll.config`, `App.config` and `Settings.settings`: remove the 4 connection strings and the 2 procedure-name settings.
7. `AllServerDataRecacheCommand` and `ManualUpdaterExtensions` recache by reloading the files.

8. Startup fails with a clear message if `Data\` or a file is missing.

Phase 6: remove SQL

1. Delete `SqlServerService`, `BaseServerService` and `IDatabaseDataService`, with their tests (including `SqlServerServiceIntegrationTests`).
2. Remove `Dapper`, `Microsoft.Data.SqlClient` and the SqlClient-driven package overrides from `Data.csproj`.
3. Rebuild the whole stack.
4. Check: `grep -rE "SqlClient|Dapper|USP_|SqlServerService"` comes back empty in all four repos.

Phase 7: tests

1. Unit tests per layer with fixture JSON files: each loader, each lookup, missing file, empty file, and case and trailing-space matching.
2. A contract test that deserializes every file in a real export into its record type.
3. A side-by-side check before cut-over: for a sample of keys, the JSON lookups return the same rows as the procedures.

Phase 8: operations and cleanup

1. Write down the refresh process: who re-runs the exporter, when, and how the files are shipped.
2. Mark the 34 procedures as no longer used by KTR in `db-object-usage-report.csv` and the `db-deprecation` scripts.

---

6. Findings and pitfalls

1. **The `CategoryType.json` that was installed before 2026-10-21 could not be read.** Its flags were the strings `"0"` and `"1"`, and Newtonsoft without `BoolJsonConverter` fails on them, so the add-in probably loaded no category types. The fresh export writes real booleans. The old file is backed up in the scratchpad.
2. **Revit 2026 has no rows in `BuiltInCategoryMeta`.** Shared parameters therefore get no category bindings on 2026, with SQL as well as with JSON.
3. **The JSON keys are the record types' property names.** The exporter maps every row into the add-in's record type (this generation's `Kelly.Windows.Data.Records`, package 26.10.21.808) without Dapper, then serializes the record. Every file loads back through `JsonFileRepository<T>` with the right row count; `EndPreparationsWelded` is checked with `JsonConvert`, because its record isn't an `IRecord`.
    - **Two names the records never matched are now renamed in the export.** `WeldLocationName` becomes `WeldLocationTerm`, and `FabricationPatternOptionTypeDotName` becomes `FabricationPatternOptionsTypeDotName`. Both properties are empty in the add-in today, so this is a behaviour change, and a fix.
    - **Still without a source column**, listed per file in `manifest.json`:
        - Missing properties: `ComponentAndLookupsViewRecord.MaterialConfigurationId`, `ProductRecord.Id`, `CustomDataUpdaterRecord.CustomData108JhkGauge` and `.Id`.
        - Columns no record maps: Terminology's `Context`, `CategoryB`, `FullName`, `InActive`, and ItemCustomData's `FilePath`.

They have their default values today too. Fix them in the procedure or the record.

    - Phase 2 should make `UspEndPreparationsWeldedSelectOne` an `IRecord`, or replace it with an `EndPreparationsWeldedRecord`, and move `BuiltInCategoryBindingRecord` from the exporter into Kelly-Windows-Data.

4. **SQL matching rules differ from C#.** The SQL `=` comparisons are case-insensitive and ignore trailing spaces. The in-memory lookups must do the same.
5. `Project.json` holds client names and addresses and is copied to every workstation. Confirm that is acceptable.
6. Measure how long the 93 MB files take to parse at startup in Revit 2024 (.NET 4.8.1). If it's too slow, write those two files without indentation, or load them lazily.